The MABEL Whole-Body Controller Suite:
Differential IK, Day-1 PD, Gravity Compensation, Self-Collision Avoidance, Admittance, Compliance, and the Swerve Base Controller
(`mabel.whole_body_controller`)

MABEL Project — Control & Teleoperation

July 24, 2026

Abstract

This document specifies every control law on the *output side* of the MABEL stack — the layer that turns desired motion (retargeted poses, app joint targets, or an operator's physical push) into safe per-actuator commands. Each law lives in its own subpackage and composes through a runtime engine: bimanual damped-least-squares differential IK with null-space posture; a plain PD position controller that runs on Day-1 hardware with *no* system identification (with a measured discrete-stability argument for why its gains must be per-joint); region-selectable gravity compensation; a capsule-based self-collision guard (velocity-damper "stop" law, CBF tracking-QP, and a torque-domain soft wall); admittance control of the mobile base driven by estimated external wrenches; region-selectable compliance ("dance with the robot"); and the delta-swerve base controller with shortest-path steering and an anti-slip cosine choke. Measured highlights: the Day-1 PD droops 0.242 rad under gravity and 0.000 rad with the gravity feedforward hook; the admittance tracks its analytic steady state $v_{ss} = F/b_l$ exactly and coasts to rest in 0.76 s; whole-body compliance glides the base under a 30 N palm pull while never leaving the tip-safe envelope. A dedicated section states and measures the *mode-invariance* of gravity compensation — selecting a control space must change the reference generator and nothing else — after a defect in which leaving Drive silently dropped the feedforward on 8 of 22 upper-body DoFs and walked the operator's palm 26.5 mm down over five space changes, now 0.7 mm.

Keywords: whole-body control, differential inverse kinematics, impedance control, admittance control, control barrier functions, self-collision avoidance, swerve drive.

1 Architecture

subpackage	law	domain
`ik`	weighted-DLS bimanual SE(3) IK + null-space posture	velocity/position
`position_controller`	per-joint PD (+ optional feedforward hook)	torque
`gravity_compensation`	$\tau = b(q, \dot{q}) - D\dot{q}$ per region	torque
`collision_avoidance`	capsule CBF / velocity damper / soft wall	both
`admittance`	wrench estimation + virtual planar mass-damper base	force→velocity
`compliance`	gravity-comp'd back-drivable region	torque
`base_controller`	swerve IK (steer + wheel speed)	velocity
`engine`	`WBCEngine`: (impedance | compliance) × region runtime	orchestration

Two hard safety rules bind every consumer: (1) base twists reach `swerve_ik` *only* through `motion_model.safe_base_step` (the tip-over gate, `motion_model.pdf`); (2) the self-collision guard is ON by default and is the *only* arm–arm defense — the MJCF disables arm–arm contact pairs, so

disabling the guard removes the physics backstop too. All torque controllers talk to the robot through the `Plant` seam (`mabel/plant.py`), so the identical code drives the MuJoCo twin and the real actuators. Dynamics quantities come from `mabel.dynamic_models` (equations derived in `dynamic_models.pdf`; not repeated here).

2 Differential IK (`ik.py`)

Both palm targets stack into a 12-dimensional task velocity $v = (v_L, \omega_L, v_R, \omega_R)$, built per palm from the pose error with proportional gains, deadbands (2 mm / 0.5°, so the arm never chases gravity sag at rest) and speed caps (1.5 m/s, 3 rad/s). With the stacked $12 \times n$ Jacobian J and a diagonal weight W ($W_{\text{lift}} = 50$, $W_{\text{torso}} = 2$, arms 1 — higher = lazier joint), the weighted damped-least-squares step is

$$\dot{q}^\star \;=\; W^{-1}J^\top\big(JW^{-1}J^\top + \lambda^2 I\big)^{-1}v \;+\; \Big(I - J^+J\Big)\,k_p^{\text{post}}(q_{\text{rest}} - q), \tag{1}$$

i.e. tracking in the weighted least-squares sense ($\min_{\dot{q}} \|J\dot{q} - v\|^2 + \lambda^2\|\dot{q}\|_W^2$, $\lambda = 0.05$) with a null-space pull toward the rest posture. An *arms-only* mode drops the lift/torso columns so manipulation never disturbs the body height. The result is post-processed by the self-collision guard (§5): the default "stop" law scales $\dot{q}^\star$ uniformly, the legacy "cbf" law re-solves the QP of Eq. (5). Per-DOF velocity boxes and joint-limit dampers bound the integration, and `apply()` integrates $q^{\text{cmd}} \mathrel{+}= \dot{q}\,\Delta t$ into the position actuators.

3 Day-1 PD position control (`position_controller/`)

Before system identification exists, the only safe torque law is model-free:

$$\tau_j \;=\; k_{p,j}\,(q_j^{\text{des}} - q_j) \;-\; k_{d,j}\,\dot{q}_j \;+\; \underbrace{\tau_j^{\text{ff}}}_{\text{optional hook}}, \qquad |\tau_j| \le \tau_j^{\max}. \tag{2}$$

Why gains must be per-joint (measured). With explicit integration at Δt the damping term alone updates $\dot{q} \leftarrow \dot{q}\,(1 - k_d\Delta t/I)$, stable only if $k_d\Delta t/I < 2$; the stiffness bound is $\omega\,\Delta t \lesssim 2$ with $\omega = \sqrt{k_p/I}$. A single gain pair that is soft on a shoulder is violently unstable on the tiny ORCA wrist: at $\Delta t = 2$ ms, $k_d = 4$ demands $I > 8 \times 10^{-3}$ kg m^2 — the wrist has $\mathcal{O}(10^{-3})$ and was measured limit-cycling at 14.7 rad/s. The controller therefore defaults to *inheriting each joint's tuned position-servo gains from the model* (k_p = `gainprm[0]`, $k_d = -$`biasprm[2]`), which are per-joint, inertia-matched and stable at the model timestep by construction; hardware plants without a model must pass explicit per-joint gains.

Day-1 → Day-2. With τ^{ff} = `region.gravity_ff()` the same controller becomes gravity-aware: the measured hold-pose droop falls from 0.242 rad to 0.000 rad (Fig. 1, k_p=25 deliberately soft to expose the droop). Targets re-seed bumplessly (`hold_current()`), and the effort clamp $\tau^{\max}$ (default 40 Nm) bounds any single-joint fault.

4 Gravity compensation (`gravity_compensation/`)

$\tau_j = b_j(q, \dot{q}) - D_j\,\dot{q}_j$ on a region's joints (bias and regions derived in `dynamic_models.pdf`; hold-vs-sag evidence there: 0.000 rad vs 1.01 rad over 2 s). Arm joints use light damping (0.6 Nm s); torso/lift

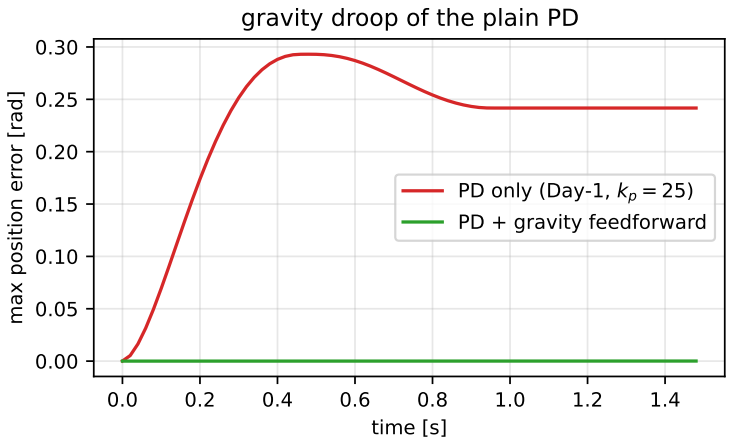


Figure 1: Gravity droop of the plain PD (Eq. (2)) holding the settled pose, with and without the gravity feedforward hook. Generated by `experiments/doc_module_figures.py`.

use "lazy" damping (12 Nm s). The controller optionally adds the guard's soft-wall torque (§5) and neutralises the managed position servos on entry. It is both a standalone mode (back-drivable float) and the feedforward source for Eq. (2).

5 Self-collision avoidance (`collision_avoidance/`)

Each protected link (forearms, hands, torso, head) is wrapped in a capsule; for a pair the surface gap is $d(q) = \|c_a - c_b\| - r_a - r_b$ with c_a, c_b the closest points of the core segments. Along the unit separation n, the gap rate is

$$\dot{d} \;=\; n^\top (J_a - J_b)\,\dot{q} \;\equiv\; a^\top \dot{q}. \tag{3}$$

Three enforcement laws share this geometry:

1. **"stop" (default): uniform velocity damper.** Scale the tracking solution $\dot{q}^\star$ by the largest $s \in [0,1]$ such that every pair's closing speed obeys

$$-\dot{d} \;\le\; \min\Big(\alpha\,(d - d_s),\; \sqrt{2\,a_{\text{stop}}\,(d - d_s)}\Big), \tag{4}$$

   the Faverjon–Tournassoud damper [1] plus a braking-distance cap (the fastest approach that can still decelerate to rest at $a_{\text{stop}} = 2\,\text{m/s}^2$ within the remaining gap — the $\sqrt{\cdot}$ term dominates at speed and absorbs the velocity-level linearization error). Pure scaling preserves direction: the hand decelerates to a stop *along the operator's intended path* — a wall, not a force field. Tangential sliding and retraction are never impeded.

2. **"cbf": tracking QP.** Impose the discrete control-barrier constraint $\dot{d} \ge -\alpha(d - d_s)$ for all monitored pairs and re-solve tracking:

$$\min_{\dot{q}} \;\tfrac{1}{2}\dot{q}^\top H \dot{q} + g^\top \dot{q} \quad \text{s.t.} \quad G\dot{q} \le h, \quad \dot{q}_{lo} \le \dot{q} \le \dot{q}_{hi}, \tag{5}$$

   with $H = J^\top J + \lambda^2 W + \mu I$, $g = -(J^\top v_{\text{des}} + \mu\,\dot{q}_{\text{post}})$ reproducing Eq. (1) when no constraint is active (Goldfarb–Idnani dual active-set via `quadprog`, OSQP and a clamp as fallbacks). This law can slide the hand *around* obstacles (the "force-field" feel) — kept as the legacy option.

3. **Torque-domain soft wall** (for compliance/gravity-comp, where there is no QP):

$$f = k_v \left[\dot{d}_{\min} - \dot{d}\right]_+ + k_p \left[d_s - d\right]_+, \qquad \tau \mathrel{+}= a\, f, \tag{6}$$

with $\dot{d}_{\min} = -\alpha(d - d_s)$: brakes only gap-closing motion and springs back out of the margin; identically zero in free motion (verified at rest in the tests).

Parameters: influence 6 cm, margin 2.5 cm, $\alpha = 6\,\mathrm{s}^{-1}$. Validated by `exp_wbc_validation.py` (guard ON: no interpenetration on adversarial crossings; OFF: interpenetrates — which is what makes the guard load-bearing) and re-checked in `tests/test_collision_avoidance.py`.

6 Admittance (`admittance/`)

Wrench sensing. Simulation reads the ground truth (`external_base_wrench`: `xfrc_applied` reduced to the chassis planar frame). Hardware uses the estimation pipeline — momentum observer → friction/joint-limit attribution → hypothesis-scoring contact localization ON the arm surface — derived in `dynamic_models.pdf` and implemented here (`wrench_estimation.py`); `ContactEstimator` reduces the wrench from the *localized* contact point, so pushing the elbow and pushing the hand both steer the base correctly. Estimation is arms-only by design: chassis/lift/torso contacts would require system identification the platform does not have. Validated against ground truth in the Control Lab (`web_control_gui/control_lab/experiments/exp_wrench_est.py`): palm pushes recover $\|F\|$ to 0.5 N, direction cosine 0.993, contact point 1.7 cm; forearm/elbow 1.1 N / cosine 0.93; rest, post-push and moving-arm negative controls all contact-free.

Base response. The chassis behaves as a virtual planar mass-damper with static (Coulomb) and viscous friction:

$$m\,\dot{v} = F - b_l\, v, \qquad I\,\dot{\omega} = M_z - b_a\,\omega, \tag{7}$$

with a static deadband ($|F| \le 3\,\mathrm{N}$, $|M_z| \le 5\,\mathrm{Nm}$ produce no motion — no creep) and a kinetic stop (unforced and slow ⇒ snap to rest — no endless roll). Steady state $v_{ss} = F/b_l$; time constant $m/b_l = 0.20\,\mathrm{s}$ ($m = 4.5\,\mathrm{kg}$, $b_l = 22\,\mathrm{N\,s/m}$, b_a high so a one-hand push yaws only gently). A comfort-box gate (referenced to the neutral *hand* pose) ramps the drive in over 4 cm of excursion so the base engages before the arm reaches its workspace limit. The output twist goes through `safe_base_step` — the admittance can never violate the tip envelope. Fig. 2 shows the measured response: the sub-deadband poke produces nothing, the 20 N step rises to exactly $v_{ss} = 0.909\,\mathrm{m/s}$, release coasts to rest in 0.76 s.

7 Compliance (`compliance/`)

Gravity compensation plus target bookkeeping: while compliant, the desired pose q^{des} continuously tracks the *measured* pose, so when the operator later switches to impedance ("Hold") the arm holds wherever it was left — no spring-back. `for_region()` matches the engine's region semantics: `arms` keeps torso, lift and base on their servos; `upper_body` adds the torso (lazy) and frees the lift; `whole_body` is realised by the engine (§9) as compliance + base admittance. Measured (`tests/test_compliance.py`): a 15 N palm push yields >0.1 rad; after release the arm settles within 0.35 rad of where it was left; the `arms` region provably leaves the lift servo untouched, `upper_body` provably frees it.

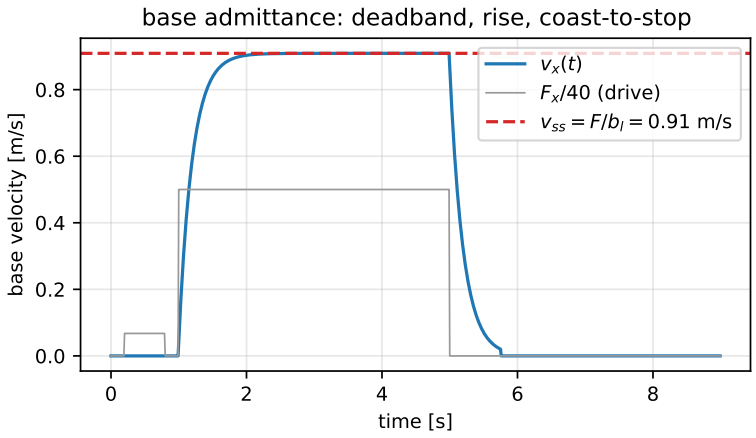


Figure 2: Base admittance (Eq. (7)): Coulomb deadband, first-order rise to $v_{ss} = F/b_l$, kinetic coast-to-stop on release.

8 Swerve base controller (`base_controller/`)

For module i at chassis offset r_i, rigid-body kinematics give the wheel velocity $v_i = (v_x - \omega r_{i,y},\ v_y + \omega r_{i,x})$; the steering target is $\theta_i = \text{atan2}(v_{i,y}, v_{i,x})$ (minus the module's mechanical mount yaw) and the wheel speed $\|v_i\|/r_w$ ($r_w = 38.1\,\text{mm}$). Two protections: *shortest-path steering* — if the steering error exceeds 90°, steer to $\theta_i \pm 180°$ and reverse the drive sign, so a column never sweeps more than ±90°; and the *anti-slip choke* — drive power is scaled by $\cos(e_\theta)$ (the velocity component the misaligned wheel can actually realise) and cut to zero above the 30° deadband, so wheels re-aim before they push (Fig. 3). This choke is the physical origin of the 0.13 s command-response ramp identified in `motion_model.pdf` §6.



Figure 3: Anti-slip drive choke vs. steering error: $\cos e_\theta$, cut at the 30° deadband.

9 The runtime engine (`engine.py`)

`WBCEngine` is what the facade drives when an app selects a torque mode. It is parameterised on two orthogonal axes — control type ∈ {`impedance` ("Hold"), `compliance` ("Soft")} and region ∈ {`arm`,

upper_body, whole_body} — and applies

$$\tau_j \;=\; \begin{cases} b_j(q,\dot{q}) + K\,(q_j^{\mathrm{des}} - q_j) - D\,\dot{q}_j + \tau_j^{\mathrm{sc}}, & j \in \mathcal{M}(\text{region}) \quad \text{(the torque law)} \\ g_j(q), & j \in \mathcal{U} \setminus \mathcal{M}(\text{region}) \quad \text{(the shared gravity layer)} \end{cases} \tag{8}$$

with K the operator-tunable stiffness (K=0 in compliance), τ^{sc} the soft wall (Eq. (6)), $\mathcal{M}$ the torque-managed set (§10) and $\mathcal{U}$ the actuated upper body of Eq. (2) in dynamic_models.pdf. The second case is what makes the engine's compensation *mode-invariant*: $\mathcal{M} \cup (\mathcal{U} \setminus \mathcal{M}) = \mathcal{U}$ for every region and control type, so the same 22 DoFs are held against gravity here as under the kinematic stack (§10). Entry/exit is *bumpless*: entering seeds q^{des} from the measured pose, neutralises exactly the managed servos (region-dependent; the lift joins in upper_body/whole_body) and snapshots the actuator gains; exit restores them byte-identically (verified in the tests). In whole-body compliance the engine adds the base loop: wrench (sim truth or estimator) → Eq. (7) → safe_base_step → swerve_ik. Measured (tests/test_compliance.py): a sustained 30 N palm pull glides the base at >0.02 m/s with tip usage ≤ 1 throughout; release coasts to <0.02 m/s; the arm region holds the base exactly still under the same pull.

10 Mode-invariant gravity compensation

The operator's control space — Drive, Task, Joint — selects a reference *generator*, and nothing else is supposed to change. Getting that wrong is an easy and long-lived class of bug, so this section states the invariant and the measurement that enforces it.

The defect (2026-07-24). "When I switch from drive to joint or task space the robot arm will sag." In the Simulation Control Center those buttons set one wire field: Drive → method="navigation", Task and Joint → method="joint" (Stiff, region arm, K = 27 Nm/rad at the default slider). Gravity compensation, however, was implemented *twice*, over two different joint sets. The kinematic stack fed all of $\mathcal{U}$; this engine called plant.begin(), which zeroes the whole qfrc_applied vector, and then re-applied effort only to $\mathcal{M}$ — the 14 arm joints. The ORCA wrists are outside the torque law by design (§9), the torso is outside it in the arm region, and the lift is outside it unless freed. So *leaving Drive silently dropped the feedforward on 8 upper-body DoFs*, and they settled to their servos' τ_g/k_p error.

The invariant. One shared layer (dynamic_models.pdf, § "One layer for every control space"), with this engine feeding exactly $\mathcal{U} \setminus \mathcal{M}$ — the second case of Eq. (8). The compensated set is then $\mathcal{U}$ in every mode by construction. tests/test_mode_switch_no_sag.py asserts the *set*, not the drift, because drift is only how the violation surfaces and it surfaces far larger on hardware, where k_p is finite.

Measurement. experiments/exp_mode_switch_sag.py settles the robot in Drive, switches space, holds, and reports per-group drift — on both deployments, because they fail differently. *In-process* (sim:mujoco, the Control Lab bench) the engine owns MjData, so servo neutralisation is real. *Split* (sim:bridge — the default, what server/run.sh runs, and the structural twin of the real robot) the gateway holds a kinematic mirror and ships (targets, effort) to the physics owner at 120 Hz; there a second half of the defect appears, namely that SimBridgePlant.advance() transmitted the effort channel only if wbc_active, so in Drive the feedforward was computed into a model the gateway never steps and then discarded — **0 of 22** compensated DoFs reaching the robot. Both halves

are fixed; the flag `MABEL_UNIFIED_GRAVCOMP=0` restores the old behaviour, which is how the "before" column below is generated by the same code.

		in-process		split (`sim:bridge`)	
quantity	group	before	after	before	after
compensated DoFs, Drive		22/22	22/22	0/22	22/22
compensated DoFs, Task/Joint		14/22	22/22	14/22	22/22
drift, Drive → Task [deg]	wrists	1.62	0.01	1.23	0.02
	neck	0.28	0.00	0.28	0.00
	torso	0.17	0.00	0.16	0.00
	arms	0.05	0.02	0.03	0.02
right palm [mm]		1.11	−0.53	0.99	−0.38
5× Drive↔Task	arms [deg]	0.24	0.08	1.58	0.12
(cumulative)	palm [mm]	2.17	1.02	26.53	−0.68

Three things are worth reading off that table. First, the in-process bench understates the fault by more than an order of magnitude: it shows 1.6 deg of wrist droop and 2.2 mm of palm travel, while the deployment the operator actually runs *ratchets* — each return to Drive sags the arms with no compensation at all, and the next Task entry re-seeds q^{des} from that sagged pose, walking the right palm 26.5 mm down over five cycles (run-to-run 22 mm to 40 mm; this loop's timing is wall-clock dependent, so the ratchet magnitude is not deterministic, unlike the single-switch numbers above it, which repeat to the reported precision). After the fix the same five cycles leave 0.7 mm. Second, the drift is independent of K across the operator's whole slider range (4.5 Nm/rad to 45 Nm/rad), which is the signature of a missing feedforward rather than a soft spring. Third, the residual is a small *rise*, not a sag: with the wrists compensated they settle nearer their commanded reference than the drooped pose Drive left them in.

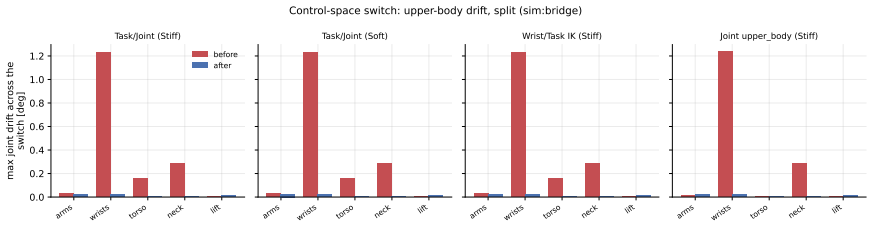


Figure 4: Upper-body drift across each control-space switch on the `sim:bridge` deployment, before and after unifying the gravity layer. Generated by `experiments/exp_mode_switch_sag.py`.

11 Experiments

Impedance step responses, push-recovery and compliance yield-and-hold are characterised in V4 of `exp_wbc_validation.py` (figure `wbcval_impedance.pdf`); self-collision in V3/V4 and `tests/test_collision_avoidance.py`; the module-level measurements of this document come from `experiments/doc_module_figures.py` and `tests/`:

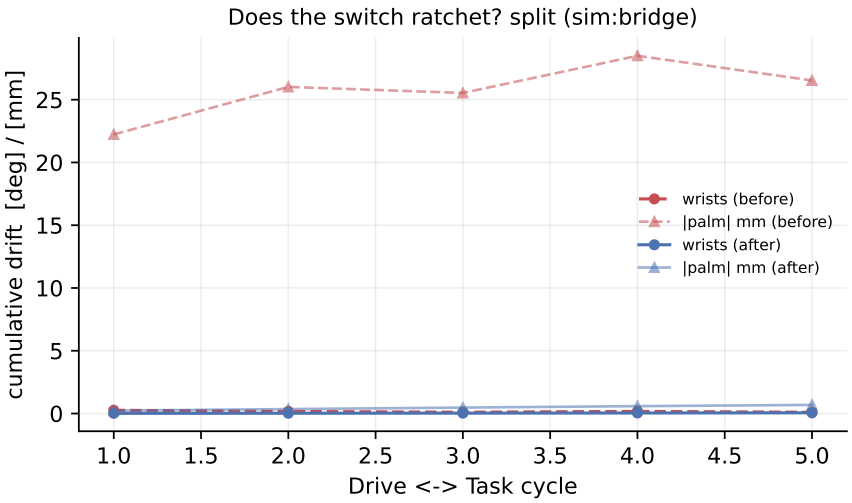


Figure 5: The ratchet: cumulative drift over repeated Drive↔Task cycles. Before the fix the error compounds because each entry adopts the already-sagged measured pose as its set-point.

law	claim	result
PD (Day-1)	step tracking, servo-inherited gains	err <0.15 rad, no limit cycle
PD	droop plain / + gravity ff	0.242 / 0.000 rad
PD	effort clamp under huge error	$\lvert\tau\rvert \le \tau^{\max}$ exactly
gravity comp	hold vs sag, 2 s	0.000 vs 1.010 rad
admittance	v_{ss} vs F/b_l	0.909 vs 0.909 m/s
admittance	deadband / coast-to-stop	no creep / 0.76 s
compliance	yield / hold after release	>0.1 / <0.35 rad
engine	whole-body glide within tip envelope	$u \le 1$ throughout
engine	exit restores servo gains	byte-identical
guard	adversarial crossing, ON / OFF	gap ≥ 0 / interpenetrates

12 Implementation & usage

```
from mabel.whole_body_controller import (WholeBodyIK, WBCEngine,
JointPositionController, GravityCompController, ComplianceController)
# Day-1 hardware:
pd = JointPositionController(plant, region_joints("arms"))
pd.set_targets({"Left Arm 3":  0.4}); pd.update()  # per tick
# app torque modes (what the facade does):
eng = WBCEngine(model, data); eng.enter("compliance", "whole_body")
eng.step(dt)  # wrench=None -> sim truth; pass an estimate on hardware
```

Legacy imports (`mabel.wbc`, `mabel.wbc_runtime`, `mabel.swerve`, `mabel.self_collision`, `mabel.admittance`) resolve here through compatibility shims.

13 Limitations

- The engine's impedance stiffness K is operator-tunable and unbounded in software; hardware deployment should clamp it against actuator limits.

- The soft wall (Eq. (6)) and the estimated-wrench admittance path are simulation-validated only; hardware validation pending.
- `quadprog`/`osqp` absence degrades the "cbf" law to a clamp (the default "stop" law is unaffected).
- The swerve choke trades command latency (0.13 s ramp) for zero scrub; aggressive gains would need the friction limit modelled.
- Guard capsules are hand-fitted at the rest pose (`SC_RADII`); a geometry change requires re-checking the rest pose is collision-free.
- §10 is verified in simulation on both deployments, but the *real* plant reaches the robot differently: `ROSPlant` does not read `qfrc_applied` at all — it publishes an arm-only feedforward from `ArmGravityCompensator` on `/arms/command.effort`, which is already mode-independent, so the physical *arms* are covered. The physical neck, torso, lift and ORCA wrists get no host-side feedforward on that path in any mode; they rely on their own servos. Closing that gap needs hardware validation and is deliberately not done blind.
- The gravity layer's recompute is throttled on a *wall* clock, so a harness running faster than real time recomputes less often per simulated second. Harmless for the sag measurements above (the robot is stationary and $g(q)$ is then constant — visible in the table as drift that does not move across the stiffness sweep), but a fast-motion study should drive it from sim time.

References

[1] B. Faverjon and P. Tournassoud, "A local based approach for path planning of manipulators with a high number of degrees of freedom," *Proc. IEEE ICRA*, 1987.

[2] A. D. Ames et al., "Control barrier functions: Theory and applications," *Proc. ECC*, 2019.

[3] B. Siciliano and O. Khatib (eds.), *Springer Handbook of Robotics*, 2nd ed., 2016 (differential IK, impedance/admittance chapters).

[4] N. Hogan, "Impedance control: An approach to manipulation," *ASME J. Dyn. Sys.*, 1985.